



Capítulo 1

Preliminares

CONCEPTS OF PROGRAMMING LANGUAGES



ROBERT W. SEBESTA

12/E

ISBN 0-321-49362-1

Tópicos do Capítulo 1

- Razões para estudar os conceitos de linguagens de programação
- Domínios de Programação
- Critérios de avaliação de idioma
- Influências no Design da Linguagem
- Categorias de idioma
- Conciliações no Design de Linguagem
- Métodos de implementação
- Ambientes de Programação

Razões para estudar os conceitos de linguagens de programação

- Maior capacidade de expressar ideias
- Melhoria na escolha de idiomas adequados
- Maior capacidade de aprender novos idiomas
- Melhor compreensão da importância da implementação.
- Melhor aproveitamento de línguas já conhecidas.
- Avanço geral da computação

Domínios de Programação

- Aplicações científicas
 - Grande número de cálculos de ponto flutuante; uso de matrizes.
 - Fortran
- Aplicações empresariais
 - Elabore relatórios, utilize números decimais e caracteres.
 - COBOL
- Inteligência artificial
 - Manipulação de símbolos em vez de números; uso de listas encadeadas.
 - LISP
- Programação de sistemas
 - Necessidade de eficiência devido ao uso contínuo.
 - C
- Software Web
 - Coleção eclética de linguagens: marcação (ex.: HTML), script (ex.: PHP), propósito geral (ex.: Java)

Critérios de avaliação de idioma

- **Legibilidade** A facilidade com que os programas podem ser lidos e compreendidos.
- **Capacidade de escrita:** a facilidade com que uma linguagem pode ser usada para criar programas
- **Confiabilidade:** conformidade com as especificações (ou seja, funciona de acordo com as especificações)
- **Custo:** o custo total final

Critérios de avaliação: Legibilidade

- Simplicidade geral
 - Um conjunto gerenciável de recursos e construções
 - Multiplicidade mínima de características
 - sobrecarga mínima do operador
- Ortogonalidade
 - Um conjunto relativamente pequeno de construções primitivas pode ser combinado de um número relativamente pequeno de maneiras.
 - Todas as combinações possíveis são legais.
- Tipos de dados
 - Tipos de dados predefinidos adequados
- Considerações sobre a sintaxe
 - Formas de identificação: composição flexível
 - Palavras e métodos especiais para formar enunciados compostos
 - Forma e significado: constructos autodescritivos, palavras-chave significativas

Critérios de avaliação: Facilidade de escrita

- Simplicidade e ortogonalidade
 - Poucas construções, um pequeno número de primitivas, um pequeno conjunto de regras para combiná-las.
- Suporte para abstração
 - A capacidade de definir e usar estruturas ou operações complexas de forma a permitir que os detalhes sejam ignorados.
- Expressividade
 - Um conjunto de maneiras relativamente convenientes de especificar operações.
 - Força e número de operadores e funções predefinidas

Critérios de avaliação: Confiabilidade

- Verificação de tipo
 - Testando erros de tipo
- Tratamento de exceções
 - Interceptar erros de tempo de execução e tomar medidas corretivas.
- Aliasing
 - Presença de dois ou mais métodos de referência distintos para o mesmo endereço de memória.
- Legibilidade e facilidade de escrita
 - Uma linguagem que não suporta formas "naturais" de expressar um algoritmo exigirá o uso de abordagens "não naturais" e, portanto, reduzirá a confiabilidade.

Critérios de avaliação: Custo

- Treinar programadores para usar a linguagem
- Escrita de programas (proximidade com aplicações específicas)
- Executando programas
- Confiabilidade: baixa confiabilidade leva a custos elevados.
- Manutenção de programas

Critérios de avaliação: Outros

- Portabilidade
 - A facilidade com que os programas podem ser migrados de uma implementação para outra.
- Generalidade
 - A aplicabilidade a uma ampla gama de aplicações.
- Bem definido
 - A completude e a precisão da definição oficial do idioma.

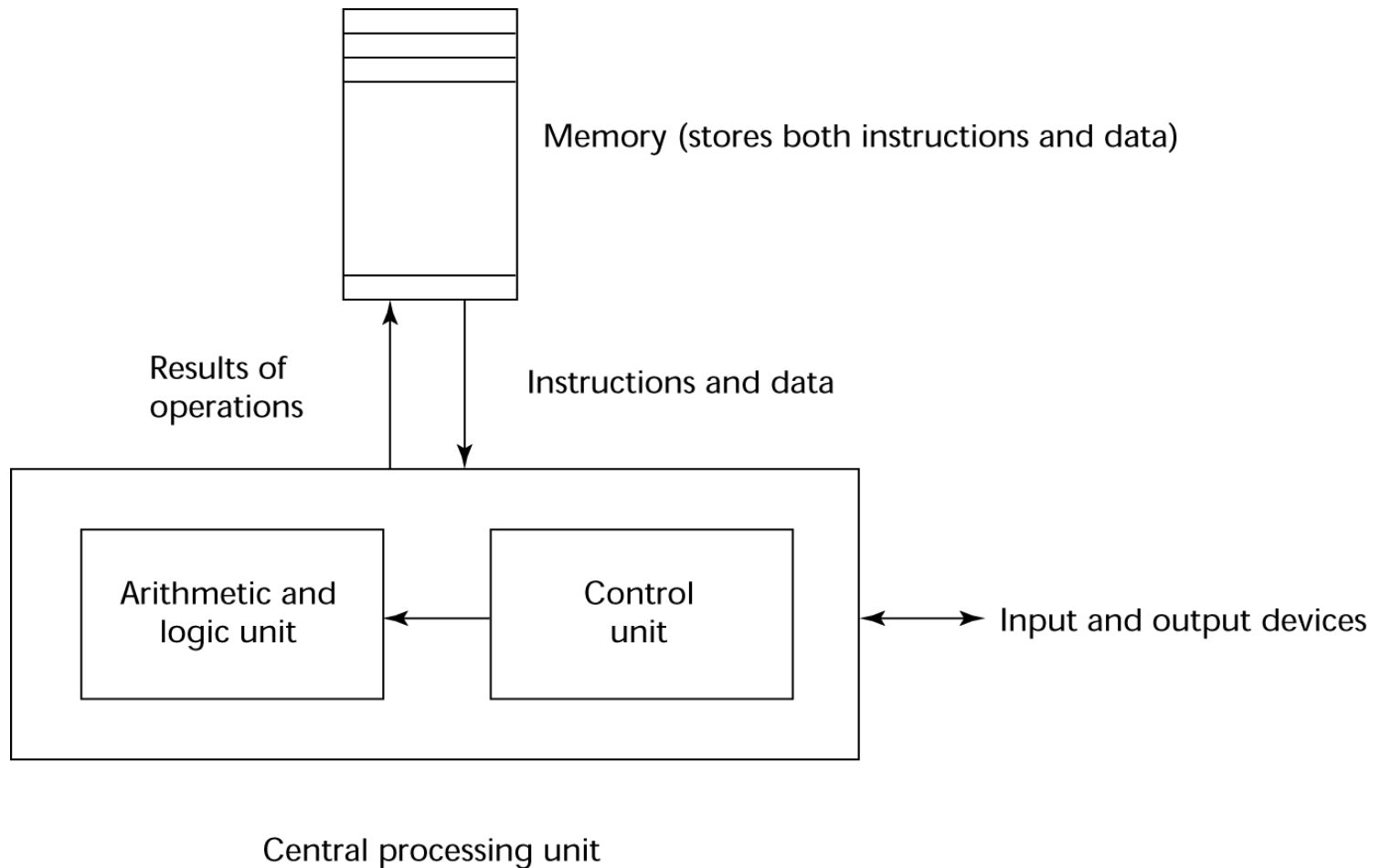
Influências no Design da Linguagem

- Arquitetura de Computadores
 - As linguagens são desenvolvidas em torno da arquitetura de computador predominante, conhecida como... arquitetura *von Neumann*
- Metodologias de projeto de programas
 - Novas metodologias de desenvolvimento de software (por exemplo, desenvolvimento de software orientado a objetos) levaram a novos paradigmas de programação e, por extensão, a novas linguagens de programação.

Influência da arquitetura de computadores

- Arquitetura de computador bem conhecida: Von Neumann
- As linguagens imperativas são as mais dominantes, devido aos computadores de von Neumann.
 - Dados e programas armazenados na memória
 - A memória é separada da CPU.
 - Instruções e dados são transmitidos da memória para a CPU.
 - Fundamentos das linguagens imperativas
 - Variáveis modelam células de memória
 - Modelo de instruções de atribuição de tubulação
 - A iteração é eficiente

A arquitetura de von Neumann



A arquitetura de von Neumann

- Ciclo de busca–execução (em um computador com arquitetura de von Neumann)

Inicialize o contador de programa.

DIA DO JUÍZO FINAL = FALSO

repita

 buscar a instrução apontada pelo contador

 incrementar o contador

 decodificar a instrução

 executar a instrução

até o DIA DO JUÍZO FINAL

Influências das Metodologias de Programação

- Décadas de 1950 e início de 1960: Aplicações simples; preocupação com a eficiência da máquina.
- Final da década de 1960: A eficiência das pessoas tornou-se importante; legibilidade, melhores estruturas de controle.
 - programação estruturada
 - projeto de cima para baixo e refinamento gradual
- Final da década de 1970: da abordagem orientada a processos para a abordagem orientada a dados.
 - abstração de dados
- Meados da década de 1980: Programação orientada a objetos
 - Abstração de dados + herança + polimorfismo

Categorias de idioma

- Imperativo
 - As principais características são variáveis, instruções de atribuição e iteração.
 - Inclua linguagens que suportem programação orientada a objetos.
 - Incluir linguagens de script
 - Inclua as linguagens visuais.
 - Exemplos: C, Java, Perl, JavaScript, Visual Basic .NET, C++
- Funcional
 - A principal forma de realizar cálculos é aplicando funções a parâmetros dados.
 - Exemplos: LISP, Scheme, ML, F#
- Lógica
 - Baseado em regras (as regras são especificadas sem uma ordem específica)
 - Exemplo: Prólogo
- Híbrido de marcação/programação
 - Linguagens de marcação estendidas para suportar alguma programação.

– Exemplos: JSTL, XSLT

Conciliações no Design de Linguagem

- **Confiabilidade versus custo de execução**
 - Exemplo: Java exige que todas as referências a elementos de array sejam verificadas quanto à indexação correta, o que leva a um aumento nos custos de execução.
- **Legibilidade versus facilidade de escrita**

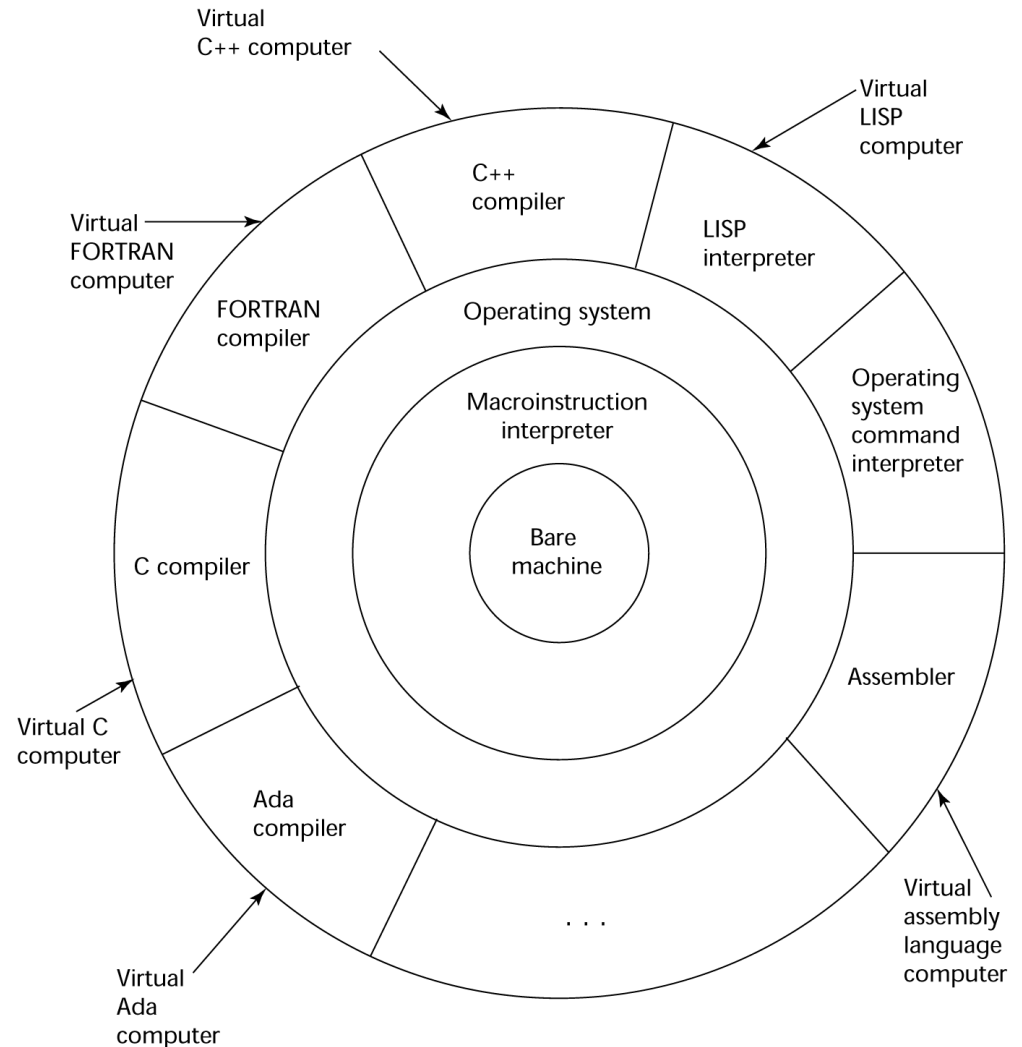
Exemplo: APL fornece muitos operadores poderosos (e um grande número de novos símbolos), permitindo que cálculos complexos sejam escritos em um programa compacto, mas ao custo de baixa qualidade. legibilidade
- **Facilidade de escrita (flexibilidade) versus confiabilidade**
 - Exemplo: Os ponteiros em C++ são poderosos e muito flexíveis, mas não são confiáveis.

Métodos de implementação

- Compilação
 - Os programas são traduzidos para linguagem de máquina; inclui sistemas JIT.
 - Uso: Grandes aplicações comerciais
- Interpretação pura
 - Os programas são interpretados por outro programa conhecido como interpretador.
 - Uso: Programas pequenos ou quando a eficiência não é um fator importante.
- Sistemas de Implementação Híbrida
 - Um meio-termo entre compiladores e interpretadores puros.
 - Uso: Sistemas de pequeno e médio porte quando a eficiência não é a principal preocupação.

Visão em camadas do computador

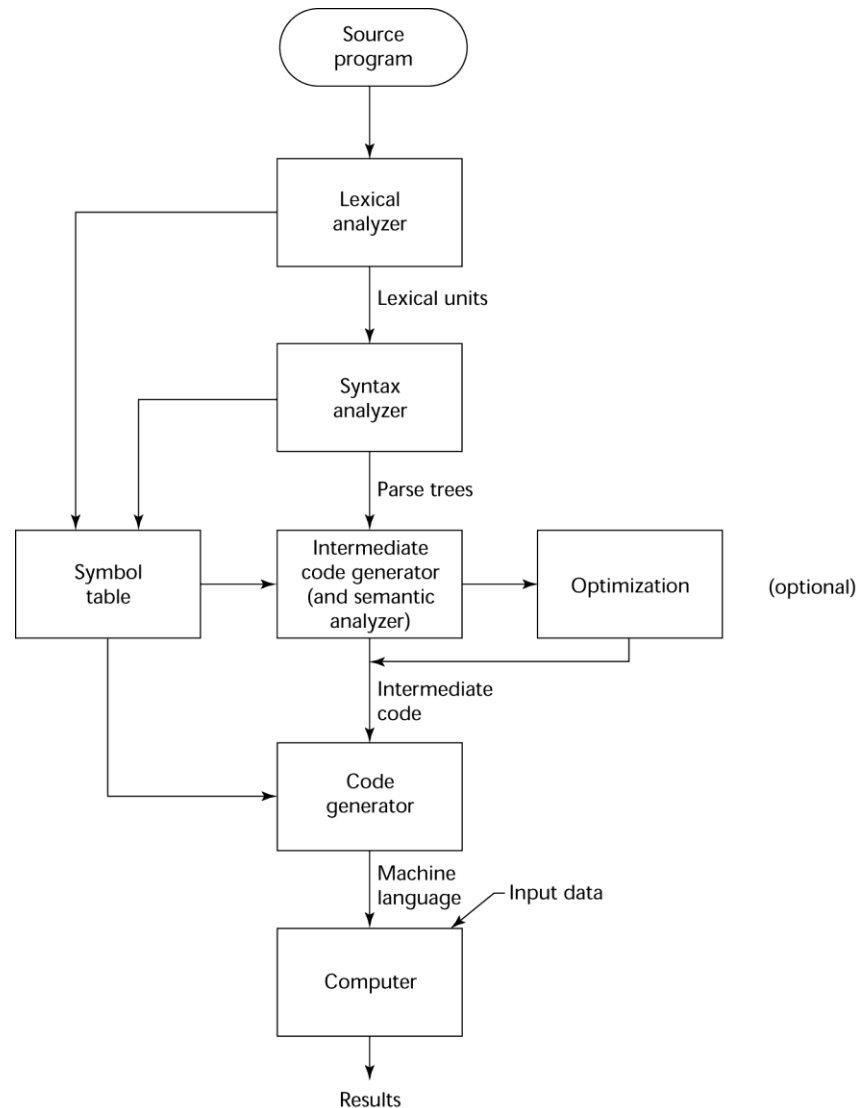
O sistema operacional e a implementação da linguagem são sobrepostos em camadas. interface de máquina de um computador



Compilação

- Traduzir um programa de alto nível (linguagem fonte) para código de máquina (linguagem de máquina).
- Tradução lenta, execução rápida
- O processo de compilação possui diversas fases:
 - Análise lexical: converte caracteres do programa fonte em unidades lexicais.
 - Análise sintática: transforma unidades lexicais em *árvores de análise sintática* que representam a estrutura sintática do programa
 - Análise semântica: gerar código intermediário
 - Geração de código: o código de máquina é gerado.

O Processo de Compilação



Terminologias de Compilação Adicionais

- **Módulo de carga**(imagem executável): o código do usuário e do sistema juntos
- **Vinculação e carregamento**O processo de coletar unidades de programa do sistema e vinculá-las a um programa do usuário.

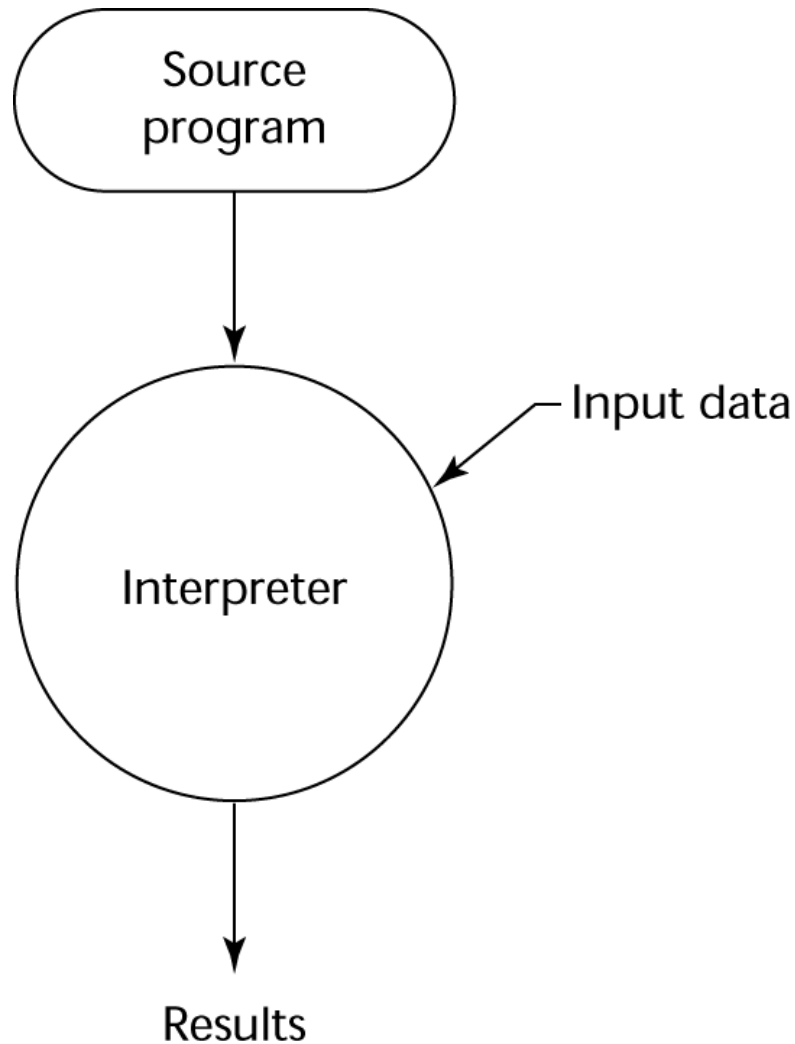
Gargalo de Von Neumann

- A velocidade de conexão entre a memória de um computador e seu processador determina a velocidade do computador.
- As instruções de um programa geralmente podem ser executadas muito mais rapidamente do que a velocidade da conexão; a velocidade da conexão, portanto, resulta em uma *gargalo*
- Conhecido como o *gargalo de von Neumann* é o principal fator limitante na velocidade dos computadores.

Interpretação pura

- Sem tradução
- Implementação mais fácil de programas (erros de tempo de execução podem ser exibidos de forma fácil e imediata)
- Execução mais lenta (de 10 a 100 vezes mais lenta que programas compilados)
- Agora é raro até mesmo em linguagens tradicionais de alto nível.
- Retorno significativo de algumas linguagens de script para a Web (por exemplo, JavaScript, PHP)

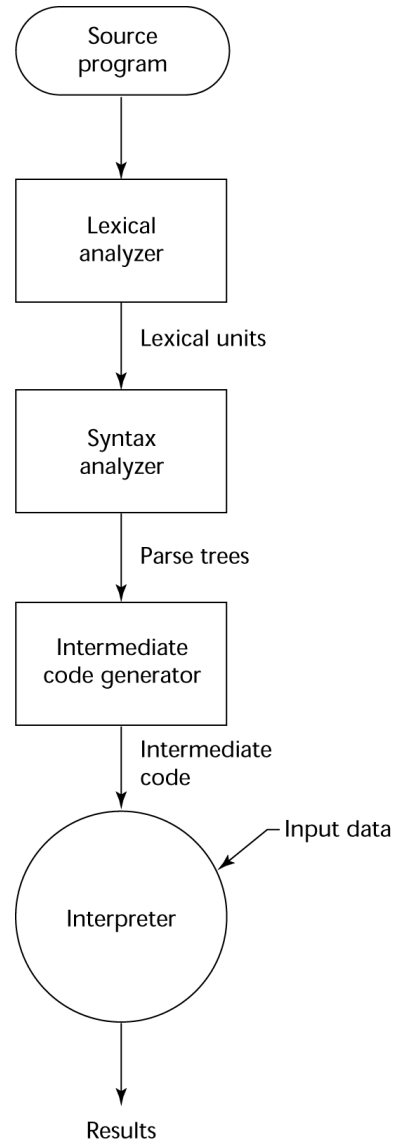
Processo de Interpretação Pura



Sistemas de Implementação Híbrida

- Um meio-termo entre compiladores e interpretadores puros.
- Um programa de idiomas de alto nível é traduzido para um idioma intermediário que permite fácil interpretação.
- Mais rápido que a interpretação pura
- Exemplos
 - Os programas em Perl são parcialmente compilados para detectar erros antes da interpretação.
 - As implementações iniciais do Java eram híbridas; a forma intermediária, *código de bytes*, proporciona portabilidade para qualquer máquina que possua um interpretador de bytecode e um sistema de tempo de execução (em conjunto, estes são chamados de *Máquina Virtual Java*)

Processo de implementação híbrida



Sistemas de Implementação Just-in-Time

- Inicialmente, traduza os programas para uma linguagem intermediária.
- Em seguida, compile a linguagem intermediária dos subprogramas em código de máquina quando eles forem chamados.
- A versão em código de máquina é mantida para chamadas subsequentes.
- Os sistemas JIT são amplamente utilizados para programas Java.
- As linguagens .NET são implementadas com um sistema JIT (Just-in-Time).
- Em essência, os sistemas JIT são compiladores com atraso.

Pré-processadores

- Macros (instruções) de pré-processador são comumente usadas para especificar que o código de outro arquivo deve ser incluído.
- Um pré-processador processa um programa imediatamente antes de o programa ser compilado para expandir as macros do pré-processador embutido.
- Um exemplo bem conhecido: o pré-processador C.
 - `expande#incluir`, `#definire` macros semelhantes

Ambientes de Programação

- Uma coleção de ferramentas utilizadas no desenvolvimento de software.
- UNIX
 - Um sistema operacional e uma coleção de ferramentas mais antigos
 - Atualmente, é frequentemente usado através de uma interface gráfica (por exemplo, CDE, KDE ou GNOME) que funciona sobre o UNIX.
- Microsoft Visual Studio.NET
 - Um ambiente visual amplo e complexo
- Utilizado para criar aplicações Web e aplicações não Web em qualquer linguagem .NET.
- NetBeans
 - Relacionado ao Visual Studio .NET, exceto para aplicativos em Java.
- Hoje
 - Multilinguagens e IA
 - Eclipse, VSCode, IntelliJ, ...

Resumo

- O estudo de linguagens de programação é valioso por diversos motivos:
 - Aumentar nossa capacidade de usar diferentes construções
 - Permita-nos escolher idiomas de forma mais inteligente.
 - Facilita o aprendizado de novos idiomas
- Os critérios mais importantes para avaliar linguagens de programação incluem:
 - Legibilidade, facilidade de escrita, confiabilidade, custo
- As principais influências no design de linguagens foram a arquitetura de máquinas e as metodologias de desenvolvimento de software.
- Os principais métodos de implementação de linguagens de programação são: compilação, interpretação pura e implementação híbrida.